SOFTWARE REQUIREMENTS SPECIFICATION

3D Volume Plane Projection & Visualization System

High-Performance Volumetric Cross-Section Renderer

DOCUMENT VERSION	TARGET ARCHITECTURE	PRIMARY INPUT	PRIMARY OUTPUT
1.0.0	CLI Executable / Script	3D NRRD Raster Data	MP4 Video Sequence

1. INTRODUCTION

1.1 Purpose & Scope

This Software Requirements Specification (SRS) details the functional, non-functional, and operational requirements for a high-performance execution script. The software ingests standard NRRD volumetric imaging datasets, parses sequential plane geometry defined via 3D point coordinates in a CSV file, performs vector-based volumetric sampling across defined plane boundaries (isosceles triangles or trapezoids), and synthesizes the resulting sequential 2D cross-sections into an MP4 video file.

1.2 Definitions & Terms

Term	Definition
3D Volume Matrix (M)	A 3D spatial grid containing unsigned 8-, 16-, or 32-bit integer voxel values representing scalar data.
NRRD	Nearly Raw Raster Data format commonly used for medical (MRI/CT) and scientific volumetric imaging.
Projection Plane (p)	A 2D spatial plane bounded in 3D space by 3 or 4 points forming an isosceles triangle or isosceles trapezoid.

2. OVERALL DESCRIPTION

Product Perspective: The application functions as an algorithmic execution tool designed for technical software engineers and scientific researchers. It operates via command-line interface to extract planar cross-sections from volumetric datasets without requiring heavy graphical user interfaces.

3. FUNCTIONAL REQUIREMENTS

3.1 Command-Line Interface Execution

REQ-CLI-1: CLI Syntax & Argument Passing

The software shall accept three positional command-line arguments upon invocation:

```
python Ex0.py <file.nrrd> <file.csv> <file.mp4>
```

3.2 Data Ingestion Requirements

REQ-DATA-1: Volumetric Matrix & Plane Parsing

The software shall parse 3D matrices (M) up to 1023^3 voxels (8/16/32-bit unsigned integers) and parse CSV plane coordinates for isosceles triangles (a,b,c) and trapezoids (a',b,a'',c).

3. FUNCTIONAL REQUIREMENTS (CONTINUED)

3.3 Algorithmic Projection & Sampling

REQ-ALG-1: Directional Vector Construction & Sampling
For each plane in the CSV, the algorithm shall calculate normalized directional vectors along plane boundaries and construct an output 2D matrix E ($D_{ab} \times D_{bc}$). Voxels shall be sampled directly from M. Any coordinate p outside volume bounds shall default to 0.

3.4 Video Export

REQ-VID-1: Frame Sequence Generation
The system shall process each plane entry in the CSV sequentially and render each 2D matrix E as an individual frame in the output MP4 video file, preserving exact CSV ordering.

4. NON-FUNCTIONAL REQUIREMENTS

REQ-PERF-1: Processing Throughput MANDATORY
On a reference 3D matrix size of 500^3 bytes, the system **must achieve a minimum processing throughput of 30 Frames Per Second (FPS)** (with an optimized target of 100+ FPS).

REQ-QUAL-1: Software Stability & Quality
The codebase must be structured cleanly with object-oriented practices (Python, C++, Java, or C) and execute without memory leaks or runtime crashes during extended frame sequences.

5. VERIFICATION & TESTING REQUIREMENTS

The system shall undergo verification across four testing levels to guarantee correctness, performance, and stability:

Test Suite ID	Focus Area	Verification Criteria & Expected Results
REQ-TEST-1	Unit Math & Sampling	Verify directional vector calculations, isosceles geometry bounds, and out-of-bounds zero-value padding ($v=o$) without array index exceptions.
REQ-TEST-2	Data Parsing	Validate parsing of various NRRD header datatypes (uint8/16/32) and line-by-line CSV coordinate extraction, including malformed line handling.
REQ-TEST-3	Performance Benchmark	Execute automated throughput benchmarking on standard 500^3 voxel volumes to confirm frame processing speed meets **≥ 30 FPS**.
REQ-TEST-4	End-to-End Pipeline	Execute complete CLI workflow (`python Ex0.py input.nrrd traj.csv output.mp4`), verifying output video playability, frame count match, and zero memory degradation.